

软件工程期末知识点复习讲义

按 Jerry 知识点复习讲义 LaTeX 模板重新整理

软件过程 - 需求工程 - UML 建模 - 设计与测试 - 维护与综合论述

使用定位：本讲义按上传资料与已识别题型重新整理，重点服务期末卷的单选、问答、综合应用和论述。内容不按教材章节平铺，而按“考点优先级 + 方法模板 + 易错点”组织。

复习方式：先看考试结构和优先级，再逐模块背结论、练模板，最后用速记清单检查名词、关系、步骤和应用边界。

资料整理：Jerry

版本：V2.0

2026 年 6 月 19 日

目录

1 总览：本课程的主线	3
2 考试结构与复习优先级	3
2.1 题型与分值结构	3
2.2 时间分配与拿分策略	4
2.3 考点优先级	4
3 模块一：软件工程概述与生命周期过程模型	5
3.1 本模块考试地位	5
3.2 核心概念	5
3.3 核心结论与适用条件	5
4 模块二：敏捷开发、团队协作与项目计划	6
4.1 本模块考试地位	6
4.2 核心概念	6
4.3 常见模型或常用结论	7
5 模块三：需求工程与 SRS	8
5.1 本模块考试地位	8
5.2 核心概念	8
5.3 核心结论与结题语言	8
6 模块四：UML 建模与面向对象分析	9
6.1 本模块考试地位	9
6.2 核心概念	9
6.3 常见关系速查	9
7 模块五：软件设计、编码与体系结构	10
7.1 本模块考试地位	11
7.2 核心概念与结论	11
7.3 体系结构风格速查	11
8 模块六：软件测试、维护与质量保障	12
8.1 本模块考试地位	12
8.2 核心概念	12
8.3 核心公式与结论	12
9 高频题型方法模板	13

10 期末高频易错点汇总	14
11 核心结论速查表	15
12 考前复习计划	15
12.1 七天版	16
12.2 三天版	16
13 考前最后 30 分钟速记清单	16
14 附录：答题用语模板	17
14.1 比较类问答	17
14.2 案例类应用	17

1 总览：本课程的主线

软件工程的考试不是单纯背概念，而是考查能否把“过程、需求、设计、测试、维护”连成一条工程化链路：

问题定义 → 需求规格 → 建模与设计 → 实现与测试 → 交付维护

本课程最终主线

- **过程主线：**可行性研究、项目计划、生命周期模型、敏捷迭代、团队协作和进度控制。
- **技术主线：**需求获取、SRS、UML 建模、体系结构设计、详细设计、编码规范、测试设计。
- **质量主线：**需求可验证，设计高内聚低耦合，测试覆盖关键风险，维护依赖文档和可修改性。

最容易混淆的概念

1. **需求分析与软件设计：**需求分析回答“系统必须做什么”，设计回答“系统怎样实现”。
2. **白盒测试与黑盒测试：**白盒依据程序内部逻辑，黑盒依据规格说明和输入输出。
3. **include 与 extend：**include 是必然发生的公共子流程，extend 是满足条件才插入的扩展流程。
4. **维护类型：**改正性修缺陷，适应性适配环境，完善性增强功能，预防性降低未来维护风险。

2 考试结构与复习优先级

2.1 题型与分值结构

题型	题量	单题分值	总分	考察特点
单选题	15 题	2 分	30 分	概念辨析、过程模型、UML、测试与维护快速判断
问答题	6 题	5 分	30 分	定义、区别、步骤、优缺点、原则类简答

综合应用题	6 题	5 分	30 分	用例图、活动图、类图、过程选择、测试用例、维护分类
论述题	1 题	10 分	10 分	围绕工程化、质量、AI 时代软件过程等进行结构化论述

2.2 时间分配与拿分策略

题型	建议用时	拿分策略
单选题	20-25 分钟	先做确定题；遇到“适合、目的、区别、不正确”等词，先定位模块再排除。
问答题	25-30 分钟	每题按“定义 - 组成/步骤 - 作用 - 注意点”写，至少写 4 个有效要点。
综合应用题	35-45 分钟	先列对象、角色、流程或测试条件，再写图形关系或用例表，避免直接画散乱图。
论述题	12-15 分钟	先给观点，再从需求、设计、实现、测试、维护和团队协作分层论证。
检查	5 分钟	检查术语、图中关系方向、测试用例有效/无效类、论述是否回扣软件质量。

2.3 考点优先级

优先级	模块	常见题型	复习要求
第一优先级	生命周期与过程模型、需求工程、UML、设计、测试	单选、问答、综合应用	必须背关键词、适用条件、步骤和典型图形关系。
第二优先级	敏捷开发、团队与项目计划、编码规范、维护	单选、问答、论述	熟悉常见比较题和案例套用题。
第三优先级	CASE 工具、质量属性、复用、再工程、数据设计	小题或应用题	会基本判断，不建议钻偏题。

第四优先级	行业趋势、课程资源、概念题或论述素材 课堂案例细节	了解即可，用于论述扩展。
-------	------------------------------	--------------

3 模块一：软件工程概述与生命周期过程模型

3.1 本模块考试地位

本模块属于：**必考核心**。它决定单选题基础分，也常作为问答题和案例题的背景。

本模块复习目标

1. 会解释软件危机、软件工程三要素和生命周期阶段。
2. 会区分瀑布、原型、增量、螺旋、V 模型、RUP 和敏捷的适用场景。
3. 会根据项目需求稳定性、风险、交付节奏选择过程模型。

3.2 核心概念

- **软件危机**：软件规模扩大后，成本、进度、质量和维护难以控制的问题集中表现。
- **软件工程**：用工程化原则、方法和工具组织软件开发与维护，目标是提高质量、降低成本、控制进度。
- **生命周期**：软件从问题定义到运行维护的全过程，常见阶段包括可行性研究、需求分析、概要设计、详细设计、编码、测试、运行维护。

3.3 核心结论与适用条件

模型	关键词	适用场景
瀑布模型	阶段顺序、文档驱动、前一阶段完成后进入下一阶段	需求稳定、技术成熟、合同和文档要求强的项目
原型模型	先做可运行样机，反复和用户确认	需求不清、用户表达困难、界面交互重要的项目
增量模型	分批交付、逐步扩展功能	需要尽早交付核心功能，后续持续扩展的项目
螺旋模型	原型 + 风险分析 + 迭代	大型、高风险、技术不确定性高的项目

V 模型	开发阶段与测试阶段对应	强调验证确认、测试计划前置的项目
RUP	用例驱动、架构中心、迭代增量	中大型面向对象项目，重视架构与用例跟踪

方法模板：过程模型选择题

1. 先判断需求是否稳定：稳定优先瀑布，不稳定优先原型或敏捷。
2. 再判断风险是否高：高风险优先螺旋。
3. 再判断是否需要快速交付：分批上线优先增量或敏捷。
4. 最后检查是否强调测试对应关系：强调测试前置和阶段对应时考虑 V 模型。

典型例题

题目：某政务系统需求文档清楚、审批流程固定、合同要求每阶段提交正式文档，应优先采用哪种过程模型？

思路：关键词是“需求清楚、流程固定、正式文档”，说明变化小且文档驱动。

答案：优先采用瀑布模型。

提醒：不要因为项目规模较大就直接选螺旋。螺旋的核心触发条件是高风险和风险分析。

本模块易错点

1. 把原型模型理解为“不要需求文档”。正确理解是用原型帮助澄清需求，最终仍要形成规格说明。
2. 把敏捷理解为“没有计划”。敏捷仍有迭代计划、站会、评审和回顾。
3. 看到“迭代”就选敏捷。RUP、螺旋和增量也有迭代特征，必须结合风险和交付方式判断。

4 模块二：敏捷开发、团队协作与项目计划

4.1 本模块考试地位

本模块属于：**高频重点**。常考敏捷宣言、Scrum 角色与制品、XP 实践、项目计划和团队沟通。

4.2 核心概念

- **敏捷开发：**强调小步快跑、持续反馈、可工作的软件和拥抱变化。

- **Scrum 三类角色**：产品负责人负责价值和产品待办列表，Scrum Master 负责过程促进和障碍清除，开发团队负责交付增量。
- **常见制品**：产品待办列表、Sprint 待办列表、产品增量。
- **常见会议**：Sprint 计划会、每日站会、Sprint 评审会、Sprint 回顾会。
- **Brooks 法则**：向已经延期的软件项目增加人手，可能使项目更晚。

4.3 常见模型或常用结论

考点	结论/做法
敏捷价值观	个体和互动高于流程和工具；可工作的软件高于详尽文档；客户合作高于合同谈判；响应变化高于遵循计划。
用户故事	通常写成“作为某类用户，我希望完成某事，以便获得某价值”。要能验收、可估算、价值明确。
燃尽图	用于观察 Sprint 剩余工作量变化，不是需求规格说明。
团队沟通	沟通路径随人数增长而快速增加，团队规模过大容易降低效率。
项目计划	明确范围、任务分解、资源、进度、风险、质量保证和交付物。

方法模板：敏捷与项目管理问答题

1. 先写概念定位，例如“敏捷强调快速反馈和持续交付”。
2. 再列角色、制品、会议或实践。
3. 结合题干说明适用条件：需求变化频繁、用户反馈强、可迭代交付。
4. 最后补充边界：敏捷不是无文档、无设计、无纪律。

本模块易错点

- Scrum Master 不是传统项目经理，也不直接决定产品需求优先级。
- 产品负责人负责产品价值和优先级，不是专门写代码的角色。
- 每日站会不是长时间技术评审，而是同步进度、计划和障碍。

5 模块三：需求工程与 SRS

5.1 本模块考试地位

本模块属于：**必考核心**。单选考概念，问答考步骤和质量特征，应用题考业务场景下的需求识别。

本模块复习目标

1. 会区分功能需求、非功能需求、业务规则和约束条件。
2. 会写需求获取、分析、规格说明、验证和管理的基本流程。
3. 会从场景中识别参与者、用例、业务事件和异常情况。

5.2 核心概念

- **需求工程**：包括需求开发和需求管理。需求开发偏“形成正确需求”，需求管理偏“控制需求变化”。
- **SRS**：软件需求规格说明，是用户、开发者、测试者共同依据的正式文档。
- **需求质量**：正确、完整、一致、无歧义、可验证、可跟踪、可修改。
- **需求跟踪**：建立需求与设计、代码、测试用例、变更记录之间的关联。

5.3 核心结论与结题语言

SRS 质量关键词

- **可验证**：测试人员能根据需求判断系统是否满足要求。
- **无歧义**：同一句需求不会被不同人员理解为不同含义。
- **一致性**：不同需求之间不互相矛盾。
- **完整性**：覆盖正常流程、异常流程、输入输出、性能和约束。

方法模板：需求场景分析题

1. 找干系人和外部系统，确认系统边界。
2. 找业务目标和触发事件，形成候选用例。
3. 拆分功能需求、非功能需求、业务规则和约束。
4. 检查需求是否可验证、完整、一致、可跟踪。

5. 对变化需求说明影响分析和变更控制流程。

典型例题

题目：在线借阅系统要求“系统响应要快，界面友好”。这句话作为需求有什么问题？应怎样改写？

思路：“快”和“友好”不可验证，属于模糊表达。

答案：可改为“在 95% 的普通查询场景下，系统应在 2 秒内返回检索结果；主要操作页面应提供清晰错误提示和返回入口”。

提醒：非功能需求也要尽量量化，否则测试无法判定是否达标。

本模块易错点

1. 把界面控件当作需求本身。按钮只是实现形式，需求应描述用户目标或系统服务。
2. 只写功能需求，忽略性能、安全、可靠性、兼容性等非功能需求。
3. 需求变更不做影响分析，导致设计、代码和测试不同步。

6 模块四：UML 建模与面向对象分析

6.1 本模块考试地位

本模块属于：**必考核心且容易拉分**。综合应用题常要求从业务描述中抽取用例、活动、类或交互关系。

6.2 核心概念

- **用例图：**表达系统边界、参与者和系统提供的服务。
- **活动图：**表达业务流程、分支、并发和责任流。
- **类图：**表达类、属性、操作和类之间的关联、聚合、组合、继承、依赖。
- **时序图：**表达对象之间按时间顺序发送的消息。

6.3 常见关系速查

关系	考试判断词	易错提醒
关联	一个对象长期知道或使用另一个对象	不要把所有“用到”都画成组合

聚合	整体与部分，部分可独立存在	弱拥有关系，生命周期不一定一致
组合	强整体与部分，部分依赖整体生命周期	整体删除后部分通常不存在
泛化	是一种，继承关系	箭头指向父类或更一般概念
include	公共子用例，必然执行	不是可选扩展
extend	条件满足时扩展基本用例	不是每次都执行

方法模板：UML 综合建模题

1. 先定系统边界，排除系统内部模块作为参与者的错误。
2. 从名词中找候选类，从动词中找候选用例或消息。
3. 给用例图标注参与者、用例、include、extend、泛化。
4. 给类图标注类名、关键属性、关键操作和关系类型。
5. 最后用一句话说明关键约束，例如权限、状态变化或异常流程。

典型例题

题目：图书借阅系统中，“续借”必须先完成“查询借阅记录”，但只有未逾期且无人预约时才能续借。应怎样表示？

答案：“续借”包含“查询借阅记录”，可用 include 表达必经公共行为；“逾期提示”或“预约冲突提示”属于条件性分支，可在活动图中用分支表示，不宜简单画成 include。

提醒：include/extend 的判断看“是否必然发生”，不是看功能是否相关。

本模块易错点

1. 把数据库、登录模块、支付模块等系统内部组成画成外部参与者。
2. 类图只写类名，不写关键属性和关系，导致答案过空。
3. 活动图忘记异常分支，例如登录失败、库存不足、权限不足。

7 模块五：软件设计、编码与体系结构

7.1 本模块考试地位

本模块属于：**必考核心**。常考设计原则、内聚耦合、体系结构风格、4+1 视图、详细设计和编码规范。

7.2 核心概念与结论

- **高内聚低耦合**：模块内部职责集中，模块之间依赖尽量少。
- **信息隐藏**：隐藏内部实现细节，只暴露稳定接口。
- **模块化**：把复杂系统分解为边界清晰、职责明确的模块。
- **抽象与逐步求精**：先确定高层结构，再逐层细化实现。
- **4+1 视图**：逻辑视图、开发视图、进程视图、物理视图、场景视图。

7.3 体系结构风格速查

风格	特点	适合场景
分层架构	上层调用下层，层次清晰	企业信息系统、教学管理系统
MVC	模型、视图、控制器分离	图形界面和 Web 应用
仓库风格	多个子系统共享中心数据仓库	编译器、数据集成系统
管道-过滤器	数据经过多个处理步骤	编译处理、数据流转换
C/S 与 B/S	客户端/服务器或浏览器/服务器	分布式业务系统
SOA	服务封装、松耦合、可组合	跨系统集成和业务复用

方法模板：设计原则与架构选择题

1. 先识别质量属性：性能、可维护性、可扩展性、安全性、可复用性。

2. 再选择结构：界面和业务分离选 MVC，层次清晰选分层，数据共享选仓库。

3. 写理由时必须同时说明优点和代价，例如分层清晰但可能增加调用开销。

4. 详细设计题要补充模块接口、数据结构、算法、异常处理和单元测试计划。

本模块易错点

• 把架构风格、设计模式、开发框架混为一谈。架构风格是系统级组织方式。

• 只写“高内聚低耦合”而不结合场景解释模块职责与依赖。

- 详细设计只写界面，不写接口、数据结构和异常处理。

8 模块六：软件测试、维护与质量保障

8.1 本模块考试地位

本模块属于：**必考核心**。单选、综合应用和论述都可能出现，尤其等价类、边界值、白盒覆盖和维护分类。

8.2 核心概念

- **测试目的**：发现缺陷、验证软件是否满足需求，而不是证明软件没有缺陷。
- **黑盒测试**：依据需求规格划分输入域和预期输出，常用等价类、边界值、因果图。
- **白盒测试**：依据程序内部结构设计测试，常用语句覆盖、判定覆盖、条件覆盖、基本路径测试。
- **维护**：软件交付后为修正错误、适应环境、改进功能或提高可维护性而进行的修改。

8.3 核心公式与结论

测试与维护速记

基本路径复杂度： $V(G) = E - N + 2P$ (P 通常为1),

简化记法： $V(G) = \text{判定节点数} + 1$.

- 等价类：每类选代表值；无效等价类通常每个测试用例只覆盖一个无效条件。
- 边界值：测边界点、边界附近点和刚越界点。
- 回归测试：修改后重新测试受影响功能，防止引入新缺陷。

方法模板：测试用例设计题

1. 列输入条件和业务规则。
2. 划分有效等价类和无效等价类。
3. 选代表值并写出预期结果。
4. 补充边界值，如最小值、最大值、刚小于、刚大于。
5. 若给程序流程图，计算独立路径数并列出基本路径。

典型例题

题目：某系统要求年龄输入为 18 至 60 的整数。请给出边界值测试思路。

答案：应覆盖 17、18、19、59、60、61，并补充非整数、空值或非法字符等无效输入。

提醒：不要只测 18 和 60，边界外侧的 17 和 61 才能验证越界处理。

本模块易错点

1. 把调试当作测试。测试是发现缺陷，调试是定位和修正缺陷。
2. 等价类只列有效类，漏掉无效类。
3. 多个无效输入放进同一个测试用例，导致缺陷定位困难。
4. 把“新增功能”归为改正性维护。新增或增强通常属于完善性维护。

9 高频题型方法模板

题型一：概念判断与单选题

识别特征：题干出现“适合、主要目的、本质、不正确、区别”。

固定步骤：

1. 圈关键词，定位模块。
2. 写出该概念的核心判别条件。
3. 用反例排除绝对化选项，例如“完全不需要文档”“证明无错误”。

易错点：敏捷不等于无计划，原型不等于最终产品，测试不等于调试。

题型二：问答题

识别特征：题干要求“简述、说明、比较、列举、为什么”。

固定步骤：

1. 第一句写定义或定位。
2. 中间列 3-5 个结构化要点。
3. 结尾写作用、适用场景或局限。

答题模板：概念 + 组成/步骤 + 优缺点 + 适用条件。

题型三：UML 建模题

识别特征：题干给业务场景、角色、操作、对象、状态变化。

固定步骤：

1. 定边界和参与者。
2. 抽取用例、类、活动或消息。
3. 标注关系：关联、聚合、组合、继承、include、extend。
4. 补异常流程和关键约束。

题型四：测试设计题

识别特征：题干给输入范围、业务规则、程序片段或流程图。

固定步骤：列条件 - 分等价类 - 选代表值 - 补边界值 - 写预期结果 - 检查无效类。

题型五：过程模型案例题

识别特征：题干给需求稳定性、风险、团队规模、交付时间、客户参与程度。

固定步骤：稳定选瀑布，需求不清选原型，高风险选螺旋，分批交付选增量，频繁变化选敏捷。

题型六：论述题

识别特征：题干包含“结合实际、你认为、评价、论述”。

固定步骤：

1. 先给明确观点。
2. 从需求、设计、实现、测试、维护、团队协作分层展开。
3. 每层至少出现一个课程术语。
4. 最后回到软件质量、成本、进度和可维护性。

10 期末高频易错点汇总

一页核心易错点

1. 需求分析写成界面设计，答案会偏离“做什么”。
2. 过程模型只背名称，不会根据需求稳定性和风险选择。
3. UML 中把内部模块画成外部参与者。
4. include 与 extend 方向和含义混淆。

5. 类图关系不写多重性或生命周期含义，表达不完整。
6. 测试用例只覆盖正常输入，漏掉无效类和边界外侧。
7. 论述题缺少课程术语，写成泛泛作文。

11 核心结论速查表

模块	核心结论	使用场景
软件工程概述	软件工程 = 方法 + 工具 + 过程，目标是质量、成本和进度可控	单选、问答开头
生命周期	问题定义、可行性、需求、概要设计、详细设计、编码、测试、维护	生命周期排序题
过程模型	稳定瀑布，不清原型，分批增量，高风险螺旋，变化敏捷	案例选择题
需求工程	需求开发 + 需求管理；SRS 要可验证、无歧义、一致、完整	问答、应用
UML 用例图	系统边界、参与者、用例、include、extend	综合应用
UML 类图	类、属性、操作、关联、聚合、组合、泛化	综合应用
设计原则	高内聚低耦合、信息隐藏、模块化、抽象、逐步求精	问答、论述
测试	黑盒看规格，白盒看结构； $V(G) = E - N + 2P$	测试题
维护	改正性、适应性、完善性、预防性	单选、论述

12 考前复习计划

12.1 七天版

7 天复习安排

1. Day 1: 过软件工程概述、软件危机、生命周期和过程模型。
2. Day 2: 复习敏捷、Scrum、团队协作和项目计划。
3. Day 3: 专项复习需求工程、SRS 和需求场景识别。
4. Day 4: 专项练 UML 用例图、活动图、类图和时序图。
5. Day 5: 专项复习设计原则、体系结构、编码规范和详细设计。
6. Day 6: 专项练测试用例、基本路径、维护分类，并做一套模拟卷。
7. Day 7: 复盘错题，背速查表和最后 30 分钟清单。

12.2 三天版

3 天冲刺安排

1. Day 1: 背过程模型、需求工程、UML 关系、测试和维护结论。
2. Day 2: 集中做单选、问答和综合应用题，整理错题模板。
3. Day 3: 按论述题模板练 1 题，最后背速记清单。

13 考前最后 30 分钟速记清单

最后 30 分钟只看这些

1. 过程模型：稳定瀑布，不清原型，分批增量，高风险螺旋，变化敏捷，测试对应看 V 模型。
2. 需求工程：需求开发 + 需求管理；SRS 要正确、完整、一致、无歧义、可验证、可跟踪。
3. UML：用例图看边界和参与者；类图看类和关系；活动图看流程；时序图看消息顺序。
4. include 必然复用；extend 条件扩展；组合生命周期依赖整体，聚合部分可独立存在。
5. 设计：高内聚低耦合、信息隐藏、模块化、抽象、逐步求精；4+1 视图不要漏场景视图。

6. 测试：黑盒看规格，白盒看结构；等价类分有效/无效；边界值测内侧、边界、外侧。
7. 维护：改正性修错，适应性适配环境，完善性加功能，预防性提升未来可维护性。
8. 论述题：观点 + 需求 + 设计 + 测试 + 维护 + 团队 + 质量收束。

14 附录：答题用语模板

14.1 比较类问答

先写二者共同点，再写差异维度：目标、输入、输出、适用场景、优缺点。不要只写定义。

14.2 案例类应用

先抄关键词，再给模型或方法选择，最后说明理由和风险。理由比结论更容易得过程分。